

Métodos Formales en la Ingeniería de Requisitos

¿Qué es un Método Formal?

"Método formal es cualquier técnica que trate la construcción y/o el análisis de modelos matemáticos que contribuyen a la automatización del desarrollo de sistemas informáticos"

Un poco de historia

- En 1967, fue Robert Floyd quien propuso utilizar lo que se denominó método de aserciones intermedias como una manera de estudiar las propiedades de los programas.
- Estas ideas fueron perfeccionadas por Hoare dando lugar al método axiomático (precondiciones E/S) donde introdujo la idea de "invariante".
- En 1976, Edsger Dijkstra, presentó un método formal llamado precondición más débil, basado en la transformación de predicados wp (weakest precondition). La idea principal era invertir los métodos de ambos de tal manera que se pudiera derivar la precondición a partir de la postcondición.

Deficiencia de los enfoques menos formales

La especificación de un sistema puede contener:

1. Contradicciones (conjuntos de sentencias que difieren entre si)
2. Ambigüedades (sentencias que se pueden interpretar de varias maneras)
3. Vaguedades (documentación voluminosa pero de información sin utilidad)
4. Sentencias incompletas (mala especificación del sistema)
5. Niveles mezclados de abstracción (imposibilita la visión global)



El papel de los métodos formales en la Ingeniería del Software

- Los métodos formales se basan en el empleo de técnicas, lenguajes y herramientas definidos matemáticamente para cumplir objetivos tales como facilitar el análisis y construcción de sistemas confiables independientemente de su complejidad, delatando posibles inconsistencias o ambigüedades que de otra forma podrían pasar inadvertidas.
- Propiedades
 - Carencia de ambigüedad
 - Consistencia (reglas de inferencia)
 - Completitud

El papel de los métodos formales en la Ingeniería del Software

- La formalización matemática del SW es el enfoque más apropiado para conseguir mejorar su calidad, va adquiriendo cada vez más fuerza.
- Los partidarios de los métodos formales defienden que su empleo, a lo largo de todo el ciclo de vida, facilita el desarrollo de especificaciones claras, concisas y no ambiguas, permite el análisis funcional de la especificación y posibilita el desarrollo de implementaciones correctas respecto a su especificación.
- Sin embargo los detractores aseguran que el empleo de métodos formales supone un volumen de trabajo considerable, aumento en los costes y tiempo de desarrollo y que debe quedar supeditado a herramientas que lo automaticen

Ventajas de los métodos formales

- Se comprende mejor el sistema.
- La comunicación con el cliente mejora ya que se dispone de una descripción clara y no ambigua de los requisitos del usuario.
- El sistema se describe de manera más precisa.
- El sistema se asegura matemáticamente que es correcto según las especificaciones.
- Mayor calidad software respecto al cumplimiento de las especificaciones.
- Mayor productividad

Problemática actual de los métodos formales

La falta de madurez en la práctica de los métodos formales es la causa de la imposibilidad de utilizarlos a nivel industrial tal y como se utilizan otros métodos de la Ingeniería del Software.

Algunas de estas causas son las siguientes:

- ❑ El desarrollo de herramientas que apoyen la aplicación de métodos formales es complicado y los programas resultantes son incómodos para los usuarios
- ❑ Los investigadores por lo general no conocen la realidad industrial.
- ❑ Es escasa la colaboración entre la industria y el mundo académico, que en ocasiones se muestra demasiado dogmático.
- ❑ Se considera que la aplicación de métodos formales encarece los productos y ralentiza su desarrollo

Clasificación de los métodos formales

- La clasificación más común se realiza en base al modelo matemático subyacente en cada método, de esta manera podrían clasificarse en:
- Especificaciones basadas en lógica de primer orden y teoría de conjuntos.
 - Los datos y relaciones/funciones se describen en detalle y sus propiedades se expresan en lógica de primer orden.
 - La semántica de los lenguajes está basada en la teoría de conjuntos.
 - Los métodos de este tipo más conocidos son: Z, VDM y B.
- Especificaciones algebraicas: proponen una descripción de estructuras de datos estableciendo tipos y operaciones sobre esos tipos.

Matemáticas en el desarrollo del software

Propiedades de las matemáticas útiles para quienes desarrollan sistemas:

- Describir de forma sucinta y exacta una situación física, un objeto o el resultado de una acción.
- Se debería presentar una especificación matemática de un sistema, y elaborar la solución en base a una arquitectura de software que implemente la especificación.
- Proporcionan una transición suave entre las actividades de ingeniería del software.
- Admite la abstracción y es un medio excelente para el modelado. Dado que es un medio exacto, hay pocas probabilidades de ambigüedad, y las especificaciones se pueden verificar matemáticamente para descubrir contradicciones e incompletitud; y, por último, la vaguedad desaparece completamente.
- Utilizar para representar niveles de abstracción en la especificación de sistema de forma organizada.

Matemáticas en el desarrollo del software

- Propiedades

- Escribir de forma clara y exacta una situación física, un objeto o el resultado de una acción.
- Proporciona una transición suave entre las actividades de ingeniería del software.
- Permite la abstracción y es excelente para el modelado.
- Hacen posible exhibir el esquema fundamental de la especificación y ayudan al analista y especificador del sistema a verificar una especificación para su funcionalidad, sin problemas tales como el tiempo de respuesta, las directrices de diseño, las directrices de implementación y las restricciones del proyecto.
- Ayuda al diseñador, porque la especificación de diseño del sistema muestra las propiedades del modelo, y ofrece tan sólo los detalles suficientes para hacer posible llevar a cabo la tarea que tengamos entre manos.
- Por último, las matemáticas proporcionan un elevado nivel de verificación cuando son usadas como medio de desarrollo del software. Cuando es preciso demostrar que un diseño se ajusta a una especificación y que algunos códigos de programa son el reflejo exacto de un diseño, es posible utilizar una demostración matemática.

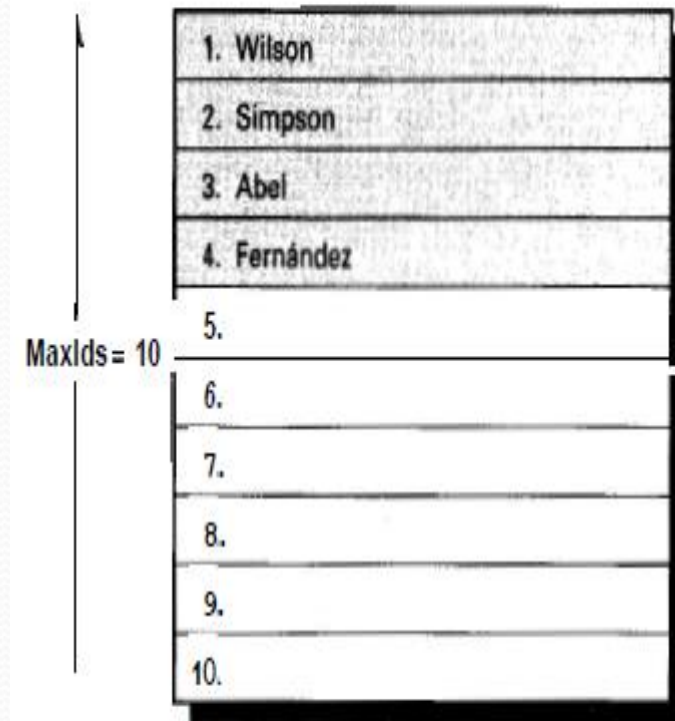
Conceptos de los métodos formales

El objetivo de esta sección es presentar los conceptos fundamentales implicados en la especificación matemática de sistema de software

Ej. Una tabla de símbolos.

1.- Un *invariante de datos* es una condición verdadera a lo largo de la ejecución del sistema que contiene una colección de datos.

El invariante de datos, que es válido para la tabla de símbolos descrita interiormente, posee dos componentes: (1) que la tabla no contendrá más de *Maxlds nombres* y (2) que no existirán nombres duplicados en la tabla.



1.	Wilson
2.	Simpson
3.	Abel
4.	Fernández
5.	
6.	
7.	
8.	
9.	
10.	

Conceptos de los métodos formales

2.- Otro concepto importante es el de *estado*. *En el contexto* de métodos formales, un estado es el dato almacenado al cual accede el sistema y que es alterado por éste.

En el ejemplo del programa de la tabla de **símbolos**, el estado es la tabla de símbolos.

3.- El concepto final es el de *operación*. *Se trata de una* acción que tiene lugar en un sistema y que lee o escribe datos en un estado. Si el programa de tabla de **símbolos** se ocupa de añadir y eliminar nombres de personal de la tabla de símbolos, entonces estará asociado a dos operaciones:

- ❑ una operación para añadir un nombre especificado en la tabla de símbolos;
- ❑ otra operación para eliminar un nombre existente de la tabla. Si el programa proporciona la capacidad de comprobar si existe o no un nombre concreto en la tabla, quiere decir que será necesaria una operación que proporcione alguna indicación de la existencia del nombre en la tabla.

Conceptos de los métodos formales

Una operación está asociada a dos condiciones: las precondiciones y las postcondiciones.

Una *precondición* define las circunstancias en que una operación en particular es válida. Por ejemplo, la precondición para una operación que añada un nombre a la tabla de símbolos de identificadores de personal es válida sólo si el nombre que hay que añadir no está almacenado en la tabla y existen menos de *Maxlds identificadores de personal* en la tabla.

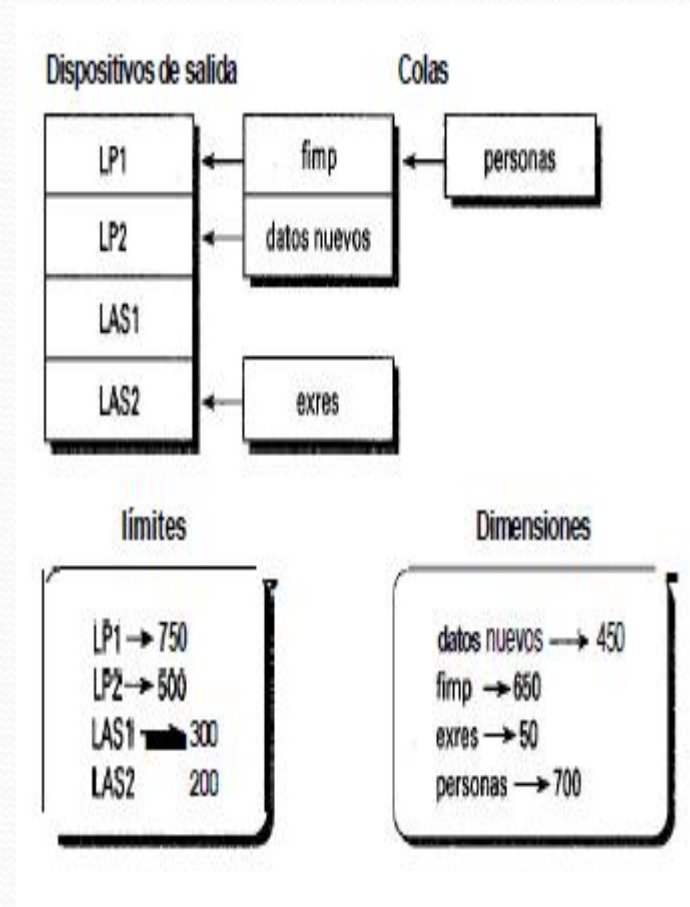
La *postcondición de una operación* define lo que ocurre cuando la operación ha finalizado su acción. Esto se define mediante su efecto sobre el estado. En el ejemplo de la operación que añade un identificador a la tabla de símbolos de identificadores de personal, la postcondición especificaría matemáticamente que la tabla habrá sido incrementada con el identificador nuevo.

Conceptos de los métodos formales

Ejemplo. Un concentrador de impresión. La parte del sistema operativo que abarca la administración de la cola de impresión.

Se supone que el sistema operativo no puede emplear más de *DispMax* dispositivos de salida .y que cada uno tiene asociada una cola.

También se supondrá que cada uno de los dispositivos está asociado a un cierto límite de líneas por archivo que se pueden imprimir.



Conceptos de los métodos formales

El invariante de datos tiene cinco componentes:

- Cada dispositivo de salida está asociado a un límite superior de líneas de impresión.
- Todo dispositivo de salida está asociado a una cola de archivos esperando impresión, que posiblemente no está vacía.
- Todo archivo tiene asociado un tamaño.
- Toda cola asociada a un dispositivo de salida contendrá archivos que tengan un tamaño menor que el límite superior de este dispositivo de salida.
- No existirán más de *DispMux* dispositivos de salida que sean administrados por este concentrador.

Conceptos de los métodos formales

El concentrador puede tener asociado un cierto número de operaciones.

Por ejemplo:

- Una operación que añada un dispositivo de salida nuevo al concentrador, junto con su límite de impresión asociado.
- Una operación que elimina un archivo de la cola asociada a un determinado dispositivo de salida.
- Una operación que añada un archivo a la cola asociada a un dispositivo de salida en particular.
- Una operación que altere el límite superior de líneas de impresión para un dispositivo de salida en particular.
- Una operación que traslade un archivo de una cola asociada a un dispositivo de salida a otra cola asociada a un segundo dispositivo de salida.

Cada una de estas operaciones se corresponde con una función del concentrador.

Conceptos de los métodos formales

Cada operación está asociada a una precondition y a una postcondition.

Ej.: La precondition para la primera operación es que el nombre del dispositivo de salida ya no existe y que existan en ese momento menos de *DispMax dispositivos de salida* a efectos del concentrador.

La postcondition es que el nombre del dispositivo nuevo se añade a la colección de nombres de dispositivos ya existentes, formándose una nueva entrada para el dispositivo sin que esta entrada tenga asociados archivos en su cola, y el dispositivo se asocia a su límite de impresión.



La precondition de la segunda operación (la eliminación de un archivo de una cola asociada a un determinado dispositivo de salida) es que el dispositivo sea conocido para el concentrador y que exista al menos una entrada en la cola asociada al dispositivo.

La postcondición es que la cabeza de la cola asociada al dispositivo de salida sea eliminada y se borre su entrada en la parte del concentrador que siga la pista de los tamaños de archivos.



La precondition de la quinta operación descrita anteriormente (el traslado de un archivo de una cola asociada a un dispositivo de salida a la cola asociada a un segundo dispositivo de salida) es:

- ❑ El primer dispositivo de salida es conocido para el concentrador.
- ❑ El segundo dispositivo de salida es conocido para el concentrador.
- ❑ La cola asociada al primer dispositivo contiene el archivo que hay que trasladar.
- ❑ El tamaño del archivo es menor o igual que el límite de impresión asociado al segundo dispositivo de salida.

La postcondición es que el archivo será eliminado de una cola y añadido a otra cola.



Preliminares Matemáticos

Conocimiento de notación matemática (conjunto, sucesiones y la notación lógica que se emplea en el cálculo de predicados).

Preliminares Matemáticos

1.- Conjuntos y especificación constructiva

Un conjunto es una colección de objetos o elementos (piedra angular).

Los elementos de un conjunto son únicos (es decir, no se permiten los duplicados).

Forman un grupo pequeño de elementos dentro de llaves, separando mediante comas sus elementos.

Por ejemplo, el conjunto

$$\{ \text{C++}, \text{Pascal}, \text{Ada}, \text{Cobol}, \text{Java} \}$$

contiene los nombres de cinco lenguajes de programación.

El orden en que aparecen los elementos dentro de un conjunto no es relevante.

Preliminares Matemáticos

El número de elementos del conjunto se conoce con el nombre de *cardinalidad*.

El operador # proporciona la cardinalidad de un conjunto.
Por ejemplo, la expresión

$$\#(A, B, C, D) = 4$$

indica que se ha aplicado el operador de cardinalidad al conjunto mostrado, con un resultado que indica el número de elementos que había en el conjunto.

Preliminares Matemáticos

Hay dos maneras de definir un conjunto.

- En primer lugar, se puede definir por enumeración de sus elementos (esta es la forma en que se han definido los conjuntos indicados anteriormente).
- El segundo método consiste en crear una *especificación constructiva de conjuntos*.

La forma general de los números de un conjunto se especifican empleando una expresión Booleana.

Las especificaciones constructivas de conjuntos resultan preferibles a las especificaciones enumeradas, porque hacen posible definir de forma sucinta los conjuntos formados por muchos miembros.

También se define explícitamente la regla que se utiliza para construir el conjunto.

Preliminares Matemáticos

Considere el siguiente ejemplo de especificación constructiva:

$$\{n : \mathbb{N} \mid n < 3 \cdot n \},$$

Esta especificación posee tres componentes:

una signatura	$n : \mathbb{N}$
un predicado	$n < 3$
y un término	n

La signatura especifica el intervalo de valores que se considerará cuando se forme el conjunto.

El predicado (una expresión Booleana) define la forma en que se debe de construir el conjunto y, por último, el término ofrece la forma general del elemento del conjunto.

En el ejemplo anterior, $n : \mathbb{N}$ denota el conjunto de los números naturales; por tanto, es preciso considerar los números naturales;

el predicado indica que solamente tienen que incluir los números naturales menores que 3;

Y el término especifica que todos los elementos del conjunto será de la forma n .

Por tanto, la especificación anterior define un conjunto: $(0, 1, 2)$

$$\{x, y : \mathbb{N} \mid x + y = 10 \cdot (x, y^2) \mid$$

2.- Operadores de conjuntos

Para representar las operaciones conjuntos y las operaciones lógicas se utiliza el mismo conjunto especializado de símbolos.

$$x \in X \quad 12 \in \{6, 1, 12, 22\}$$

$$x \notin X \quad 13 \notin \{13, 1, 124, 221\}$$

$$A \subset B \quad \{1, 2\} \subset \{4, 3, 1, 2\}$$

$$\{HD1, LP4, RC5\} \subset \{HD1, RC2, HD3, LP1, LP4, LP6\}$$

$$\{HD1, LP4, RC5\} \subseteq \{HD1, RC2, HD3, LP1, LP4, LP6\}$$

$$\{HD1, LP4, RC5\} \subseteq \{HD1, LP4, RC5\}$$

Un conjunto especial es el conjunto vacío $\emptyset$. En matemáticas normales este operador se corresponde con el cero. El conjunto vacío tiene la propiedad de que es un subconjunto de todos los conjuntos restantes.

$$\emptyset \cup A = A \text{ y } \emptyset \cap A = \emptyset$$

$\{\text{Archivo1}, \text{Archivo2}, \text{Impuesto}, \text{Compilador}\} \cup$
 $\{\text{NuevoImpuesto}, \text{D2}, \text{D3}, \text{Archivo2}\}$

$\{\text{Archivo1}, \text{Archivo2}, \text{Impuesto}, \text{Compilador},$
 $\text{NuevoImpuesto}, \text{D2}, \text{D3}\}$

$\{12, 4, 99, 1\} \cap \{1, 13, 12, 771\}$

$\{12, 1\}$.

El operador diferencia de conjuntos “\” como el mismo nombre indica, forma un conjunto eliminando los elementos del segundo operando de entre los elementos del primer operando.

$$\{\text{Nuevo, Viejo, ArchivoImpuesto, ParamSistema}\} \setminus \{\text{Viejo, ParamSistema}\}$$
$$\{\text{Nuevo, ArchivoImpuesto}\}.$$
$$\{a, b, c, d\} \cap \{x, y\}$$

El operador final es el *producto cruzado* **x**, **conocido** algunas veces también como *producto cartesiano*, el cual posee dos operandos que son conjuntos de parejas.

El resultado es un conjunto de parejas en donde cada una se compone de un elemento tomado del primer operando, combinado a su vez con un elemento del segundo operando

$$\{1,2\} \times \{4,5,6\}$$

$$\{(1,4), (1,5), (1,6), (2,4), (2,5), (2,6)\}$$

Un concepto importante en los métodos formales es el operador *conjunto potencia*. *Se trata de la colección de subconjuntos de ese conjunto.*

Este es un operador unario que cuando se aplica a un conjunto devuelve el conjunto de subconjuntos del operando.

$$\mathcal{P} \{1, 2, 3\} = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$$

Preliminares Matemáticos

Operadores lógicos

Otro componente importante de los métodos formales es la lógica: el álgebra de las expresiones verdaderas y falsas.

Los operadores matemáticos equivalentes son los siguientes:

$\wedge$	y
$\vee$	o
$\neg$	no
$\Rightarrow$	implica

La *cuantificación universal* es una manera de *confeccionar* una afirmación acerca de los elementos del conjunto que resulta ser verdadera para todos los miembros de ese conjunto.

$$\forall i, j : \mathbb{N} \cdot i > j \Rightarrow i^2 > j^2$$

4.- Sucesiones

Una sucesión es una estructura matemática que modela el hecho consistente en que sus elementos **están ordenados**.

Una sucesión **S** es un conjunto de parejas cuyos elementos oscilan entre *1* y el *elemento de mayor número*.

Los objetos que forman los primeros elementos de las parejas se conocen como *dominio de una sucesión*, y la colección de segundos elementos como *intervalo de la sucesión*.

Una sucesión vacía se representa mediante la forma ... $\langle \rangle$

A las sucesiones se les puede aplicar todos los operadores de conjuntos

$$\{ (1, \text{Jones}), (2, \text{Wilson}), (3, \text{Shapiro}), (4, \text{Estévez}) \}$$

(Jones, Wilson, Shapiro, Estévez)

(Jones, Wilson, Shapiro) $\neq$ (Jones, Shapiro, Wilson)

La concatenación, **es un operador binario que forma una sucesión que** se construye añadiendo el segundo operando al final de su primer operando.

$$(2, 3, 34, 1) \wedge (12, 33, 34, 200)$$

Otros operadores que pueden aplicarse a las sucesiones son *cabeza*, *cola*, *frente* y *último*. **El operador cabeza extrae el primer elemento de una sucesión**; el operador *cola* proporciona los últimos $n-1$ elementos finales de una sucesión de longitud n ; por *Último* el operador *Último* extrae el elemento final de una sucesión; y el operador *frente* proporciona los últimos $n-1$ elementos en una sucesión de longitud n .

```
cabeza(2, 3, 34, 1, 99, 101)= 2  
cola(2, 3, 34, 1, 99, 101)=⟨3, 34, 1, 99, 101⟩  
último(2, 3, 34, 1, 99, 101)= 101  
frente(2, 3, 34, 1, 101)=⟨2, 3, 34, 1, 99⟩
```

Cuando en un estado se utiliza una sucesión, se debería designar utilizando la palabra clave *seq*.

```
ListaArchivos: seq ARCHIVOS  
NingúnUsuario: \N
```


Lenguajes Formales de Especificación

Un lenguaje de especificación formal suele estar compuesto de tres componentes primarios:

- ❑ una sintaxis que define la notación específica con la cual se representa la especificación;
- ❑ una semántica que ayuda a definir un «universo de objetos» que se utilizará para describir el sistema;
- ❑ un conjunto de relaciones que definen las reglas que indican cuáles son los objetos que satisfacen correctamente la especificación.



El *dominio sintáctico de un lenguaje de especificación* formal suele estar basado en una sintaxis derivada de una notación estándar de la teoría de conjuntos y del cálculo de predicados. (Ej.: las variables tales como x , y , y z describen un conjunto de objetos que están relacionados a un problema y se utilizan junto con los operadores).

Aunque la sintaxis suele ser simbólica, también se pueden utilizar iconos (símbolos gráficos como cuadros, flechas y círculos) si no son ambiguos.

El *dominio semántico de un lenguaje de especificación* indica la forma en que ese lenguaje representa los requisitos del sistema.

Por ejemplo, un lenguaje de programación posee un conjunto de semánticas formales que hace posible que el desarrollador de software especifique algoritmos que transforman una entrada en una salida. Sin embargo, un lenguaje de programación no es un buen lenguaje de especificación, porque solamente puede representar funciones computables. Un lenguaje de especificación deberá poseer un dominio semántico más amplio; esto es, el dominio semántico de un lenguaje de especificación debe de ser capaz de expresar ideas tales como «Para todo x de un conjunto infinito A , existe un y de un conjunto infinito B tal que la propiedad P es válida para x e y ».

- 
- Es posible utilizar distintas abstracciones semánticas para describir un mismo sistema de diferentes maneras.
 - En la actualidad se utiliza toda una gama de lenguajes formales de especificación: CSP, VDM y Z son lenguajes formales de especificación representativos que muestran las características indicadas anteriormente.

Uso del lenguaje Z

Las especificaciones en Z se estructuran como un conjunto de esquemas.

Un esquema es, en esencia, una especificación formal análoga a la subrutina o el procedimiento de un lenguaje de programación.

Del mismo modo que los procedimientos y **las subrutinas se utilizan** para estructurar un sistema, los esquemas se utilizan para estructurar una especificación formal.

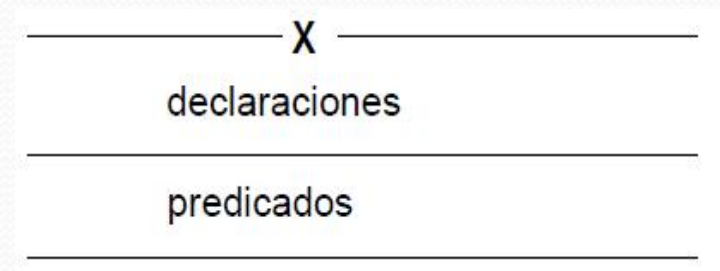
La notación Z está basada en la teoría de conjuntos con tipos y **en la lógica de primer orden. Z proporciona** una estructura denominada ***esquema***, ***para describir el estado y las operaciones de una especificación.***

Los esquemas agrupan las declaraciones de variables con una lista de predicados que limitan los posibles valores de las variables.

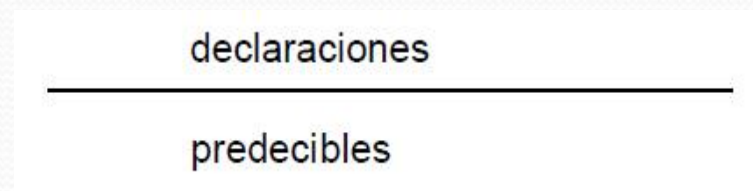
Son estructuras parecidas a cuadros que presentan variables y que especifican la relación existentes entre las variables.

Un esquema es una especificación formal análoga a una subrutina (procedimiento).

En Z ***el esquema X se define en la forma***



Las funciones constantes globales se definen en la forma



- La declaración proporciona el tipo de la función o constante, mientras que el predicado proporciona su valor.

Conjuntos:	
$S: \mathbb{N} \times X$	S se declara como un conjunto de X.
$x \in S$	x es miembro de S.
$x \notin S$	x no es miembro de S
$S \subseteq T$	S es un subconjunto de T: Todo miembro de S está también en T.
$S \cup T$	La unión de S y T: Contiene todos los miembros de S o T o ambos.
$S \cap T$	La intersección de S y T: Contiene todos los miembros tanto de S como de T.
$S \setminus T$	La diferencia de S y T: Contiene todos los miembros de S salvo los que están también en T.
$\emptyset$	Conjunto vacío: No contiene miembros.
$\{x\}$	Conjunto unitario: Solamente contiene a x.
$\mathbb{N}$	El conjunto de los números naturales 0, 1, 2,
$S: \mathbb{F} \times X$	Se declara S como un conjunto finito de X.
$\max(S)$	El máximo del conjunto no vacío de números S.
Funciones:	
$f: X \rightarrow Y$	Se declara como una inyección parcial de X a Y.
$\text{dom } f$	El dominio de f: Dícese del conjunto de valores de x para los cuales está definido $f(x)$.
$\text{ran } f$	El rango de f: El conjunto de valores que toma $f(x)$ cuando x recorre el dominio de f.
$f \oplus \{x \mapsto y\}$	Una función que coincide con f salvo que x se hace corresponder con y.
$\{x\} \triangleleft f$	Una función igual que f, salvo que x se ha eliminado de su dominio.
Lógica:	
$P \wedge Q$	P y Q: Es verdadero si tanto P como Q son verdaderos.
$P \Rightarrow Q$	P implica Q: Es verdadero tanto si Q es verdadero como si P es falso.
$\theta S' = eS$	Ningún componente del esquema S cambia en una operación.

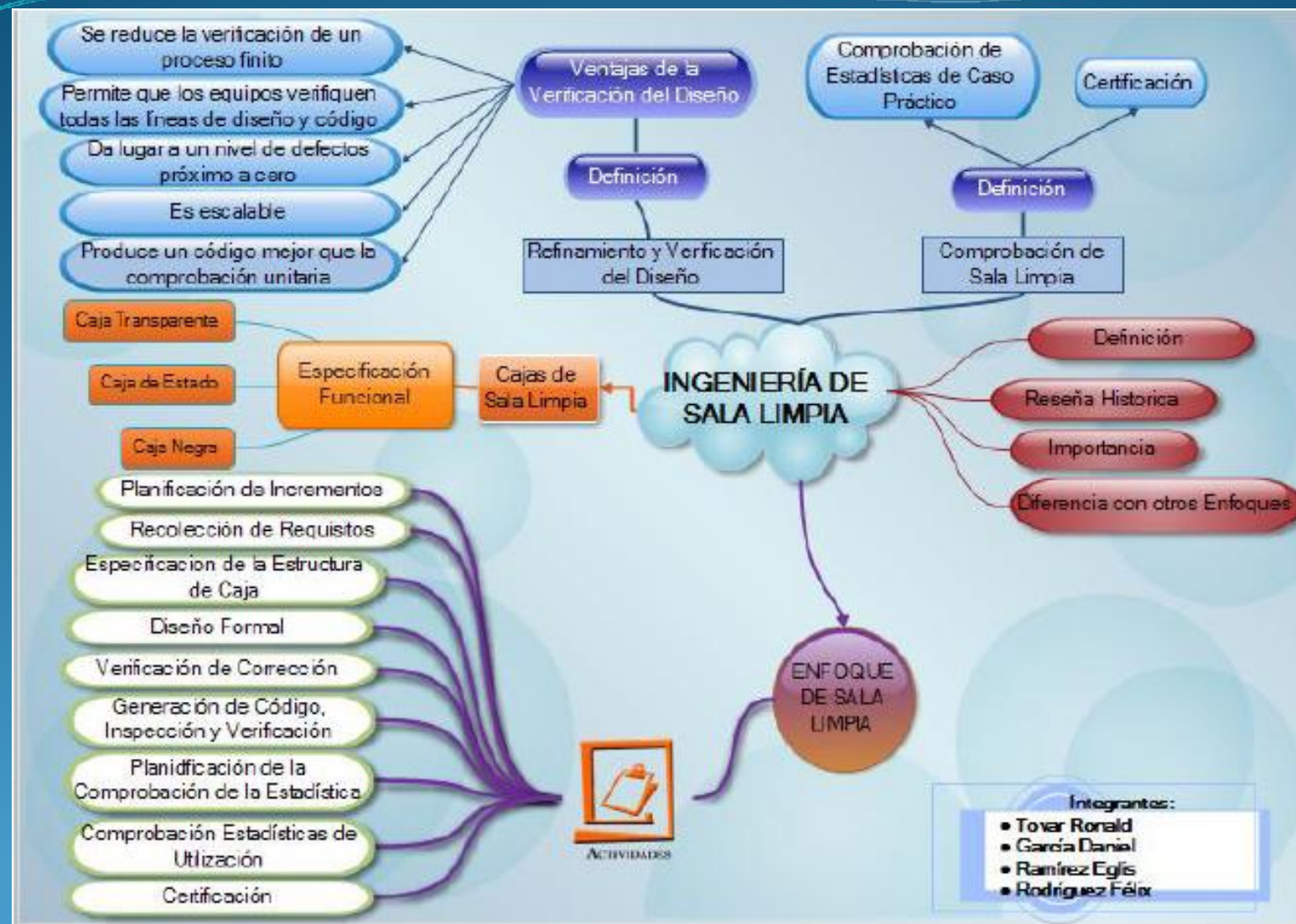
Reglas para aplicar metodos formales

- **Seleccionarás la notación adecuada.**
- **Formalizarás, pero no de más.**
- **Estimarás los costes**
- **Poseerás un experto en métodos formales a tu disposición.**
- **No abandonarás tus métodos formales de desarrollo.**
- **Documentarás suficientemente.**
- **No comprometerás los estándares de calidad**
- **No serás dogmático**
- **Comprobarás, comprobarás y volverás a comprobar**
- **Reutilizarás cuanto puedas**

El futuro de los Metodos Formales

- Aun cuando las técnicas de especificación formal, con fundamento matemático, todavía no se utilizan con demasiada frecuencia en la industria) éstas ofrecen ciertamente unas ventajas substanciales con respecto a las técnicas menos formales.
- Las especificaciones formales se pueden estudiar matemáticamente, mientras que las especificaciones informales no pueden estudiarse de esta manera.
- Además, la especificación formal elimina la ambigüedad, y propugna un mayor rigor en las primeras fases del proceso de ingeniería del software.
- La especificación formal se centra fundamentalmente en las funciones y los datos. La temporización, el control y los aspectos de comportamiento del problema son más difíciles de representar.
- Existen ciertas partes del problema (por ejemplo, las interfaces hombre-máquina) que se especifican mejor empleando técnicas gráficas.
- La especificación mediante métodos formales es más difícil de aprender que otros métodos de análisis

Ingeniería del Software de sala Limpia





La filosofía que subyace tras la ingeniería del software de sala limpia consiste en evitar la **dependencia** de costosos procesos de eliminación de defectos mediante la escritura de incrementos de código desde un primer momento, y mediante la verificación de su corrección antes de las pruebas. Su modelo de proceso incluye la certificación estadística de calidad de los incrementos de código, a medida que estos se van añadiendo en el sistema.

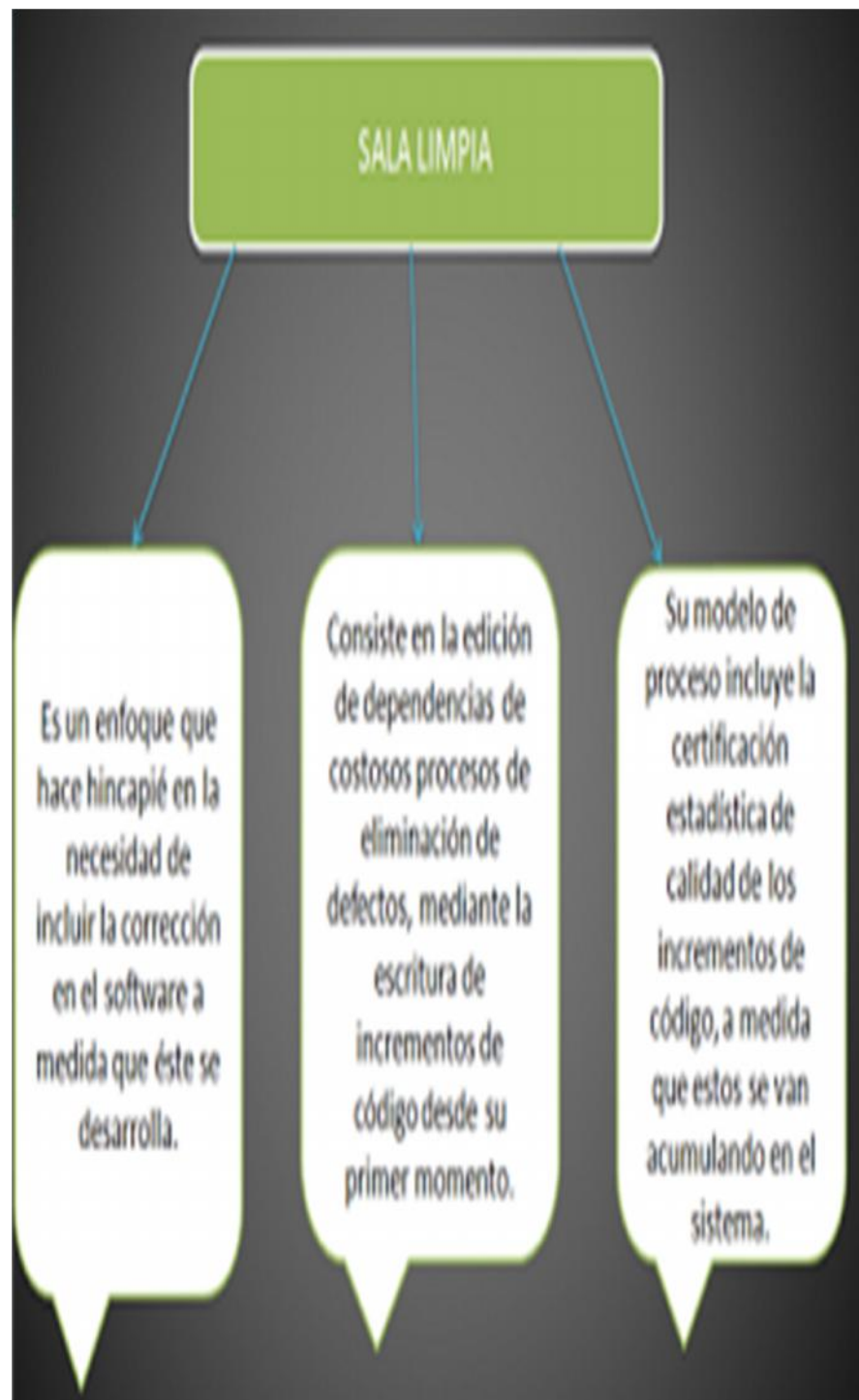
La utilización integrada de IS convencional, métodos formales, verificación de programas (demostración de corrección) y estadística SQA se combinan en una técnica que pueda producir un producto de calidad

EL ENFOQUE DE SALA LIMPIA

La filosofía de la «sala limpia» en las técnicas de fabricación de hardware es en realidad algo bastante sencillo:

Se trata de una forma rentable y eficiente, en términos de tiempo, de establecer un enfoque de fabricación que impida la introducción de defectos de producción.

En lugar de fabricar un producto y dedicarse después a eliminar defectos, el enfoque de sala limpia demanda la disciplina necesaria para eliminar errores en las especificaciones y en el diseño, fabricando entonces el producto de forma «limpia».



Propuesta por primera vez para la ingeniería del software por parte de Mills y sus colegas durante los años 80.

A hoy no a alcanzado una máxima utilización. Posibles razones:

La creencia en que la metodología de sala limpia es excesivamente teórica, matemática y excesivamente radical para utilizarla en el desarrollo de software real.

No propugna una comprobación unitaria por parte de los desarrolladores, sino que la sustituye por una verificación de la corrección y por un control estadístico de la calidad.

La madurez de la industria de desarrollo del software. El uso de procesos de sala limpia requiere una aplicación rigurosa de procesos definidos en todas las fases del ciclo de vida.



Aun cuando existen ciertos indicios de verdad en todas las indicaciones expresadas anteriormente, los posibles beneficios de la ingeniería del software de sala limpia compensan más que sobradamente la inversión requerida para superar la resistencia cultural que se encuentra en el núcleo de estas objeciones.

La estrategia de sala limpia

El enfoque de sala limpia hace uso de una versión especializada del modelo incremental de software .

Se desarrolla un «cauce de incrementos de software» por parte de equipos de ingeniería del software pequeños e independientes.

A medida que se va certificando cada incremento, se integra en el todo. Consiguientemente, la funcionalidad del sistema va creciendo con el tiempo.





Se producen las tareas siguientes:

- **Planificación de incrementos.**
- **Recolección de requisitos.**
- **Especificación de la estructura de cajas.**
- **Diseño formal.**
- **Verificación de corrección.**
- **Generación de código, inspección y verificación.**
- **Planificación de la comprobación estadística.**
- **Comprobación estadística de utilización.**
- **Certificación.**

Planificación de incrementos

Se desarrolla un plan de proyecto que adopta la estrategia incremental. Se van estableciendo las funcionalidades de cada uno de los incrementos, su tamaño estimado y un plan de desarrollo de sala limpia. Es preciso tener especial cuidado para asegurar que los incrementos certificados se vayan integrando de forma temporalmente oportuna.

Recolección de requisitos

Mediante el uso de técnicas, se desarrolla una descripción más detallada de requisitos del nivel del usuario (para cada incremento).

Especificación de la estructura de cajas

- Se utiliza un método de especificación que hace uso de estructuras de caja para describir la especificación funcional.
- Ajustado a los principios de análisis operacional, **la estructura de caja «aísla y separa la definición creativa del comportamiento, de los datos, y de los procedimientos para cada nivel de refinamiento».**

Diseño formal

Mediante el uso del enfoque de estructura de cajas, el diseño de sala limpia es una extensión natural y sin discontinuidades de la especificación.

Aun cuando es posible efectuar una distinción clara entre estas dos actividades, las especificaciones (que se denominan “cajas negras” se refinan iterativamente (dentro de cada incremento) para transformarse en diseños análogos a la arquitectura y a los procedimientos se denominan «cajas de estado» y «cajas transparentes», respectivamente).

Verificación de corrección

El equipo de sala limpia lleva a cabo una serie de rigurosas actividades de verificación de corrección aplicadas primero al diseño y después al código.

La verificación comienza con la estructura de cajas del más alto nivel (la especificación) y avanza hacia el detalle de diseño y el código.

El primer nivel de verificación de corrección se lleva a cabo aplicando un conjunto de cuestiones de corrección. Si este conjunto de preguntas no demuestra que la especificación es correcta, se utilizan métodos más formales (matemáticos) de verificación.

Generación de código, inspección y verificación

Las especificaciones de estructura de caja, que se representan mediante un lenguaje especializado, se traducen al lenguaje de programación adecuado.

Se utilizan entonces técnicas estándar de recorrido o de inspección para asegurar el cumplimiento semántico de las estructuras de código y de cajas, y la corrección sintáctica de código.

A continuación, se efectúa una verificación de corrección para el código fuente.

Planificación de la comprobación estadística

La utilización estimada del software se analiza, se planifica y se diseña un conjunto de casos de prueba que ejerciten la «distribución de probabilidad» de esa utilización, esta actividad de sala limpia se realiza en paralelo con la especificación, la verificación y la generación de código.

Comprobación estadística de utilización

Recordando que es imposible una comprobación exhaustiva del software de computadora, siempre resulta necesario diseñar un conjunto finito de casos de prueba.

Las técnicas estadísticas de utilización ejecutan una colección de pruebas derivadas de una muestra estadística de todas las posibles ejecuciones del programa por parte de todos los usuarios de una cierta población objetivo.

Certificación

Una vez que se ha finalizado la verificación, la inspección y la comprobación de utilización (y después de corregir todos los errores) se certifica el incremento como preparado para su integración.

Al igual que otros modelos de proceso del software, el proceso de sala limpia hace especial hincapié en la necesidad de conducir unos modelos de análisis y de diseño de muy alta calidad.

La notación de estructura de cajas no es más que otra forma para que el ingeniero del software pueda representar los requisitos y el diseño.

La distinción real del enfoque de sala limpia consiste en que se aplica la verificación formal a los modelos de ingeniería.

Qué hace diferente la sala limpia

La sala limpia representa el primer intento práctico de poner el proceso de desarrollo del software bajo un control estadístico de calidad con una estrategia bien definida para la mejora continua del proceso.

Para alcanzar esta meta, se definió un ciclo único de vida de sala limpia, que hacía hincapié en una ingeniería del software basada en las matemáticas para obtener diseños de software correctos y que se basaba en software basado en estadística para *la certificación* de fiabilidad de ese software.



La ingeniería del software de sala limpia difiere de los puntos de vista convencionales y orientados a objetos porque:

1. Hace uso explícito del control estadístico de calidad.
2. Verifica la especificación del diseño empleando una demostración de corrección basada en las matemáticas.
3. Hace mucho uso de la comprobación estadística de utilización para descubrir errores de especial incidencia.

La ingeniería de sala limpia diverge de las prácticas de software convencionales al quitar importancia al papel de las pruebas de unidad y a la depuración y al reducir dramáticamente (o eliminar) la cantidad de comprobaciones que son realizadas por quien desarrolla el software.

En el desarrollo convencional del software, los errores se aceptan como cosas que pasan.

Dado que se considera que los errores son inevitables, cada módulo del programa debe ser comprobado unitariamente (para descubrir los errores) y depurado después (para eliminar los errores).

Cuando se publica finalmente el software, la utilización práctica descubre aun más defectos, y comienza otro ciclo de comprobación y depuración.

Este trabajo repetido asociado a las actividades mencionadas resulta costoso y lleva mucho tiempo. Y lo que es peor, puede ser degenerativo; la corrección de errores puede (inadvertidamente) dar lugar a la introducción de otros errores.

En la ingeniería del software de sala limpia, la comprobación unitaria y la depuración se ven sustituidas por una verificación de corrección y por pruebas basadas en la estadística. Estas actividades, junto con el mantenimiento de registros para una continua mejora, hacen que el enfoque de sala limpia sea único.

Especificación Funcional

Independientemente del método de análisis seleccionado, los principios de operación siempre serán aplicables.

Se modelan los datos, las funciones y el comportamiento.

Los modelos resultantes deben de ser descompuestos (refinados) para proporcionar un grado de detalle cada vez más elevado.

El objetivo global consiste en pasar de una especificación que captura la esencia de un problema, a una especificación que proporciona una cantidad de detalle sustancial para su implementación.

La ingeniería del software de sala limpia satisface los principios de análisis operacional por cuanto emplea un método denominado *especificación de estructura de caja*.

Una caja encapsula el sistema (o algún aspecto del sistema) con un cierto grado de detalle. Mediante un proceso de refinamiento progresivo, se van refinando las cajas para formar una jerarquía en la cual cada caja tiene transparencia referencial.

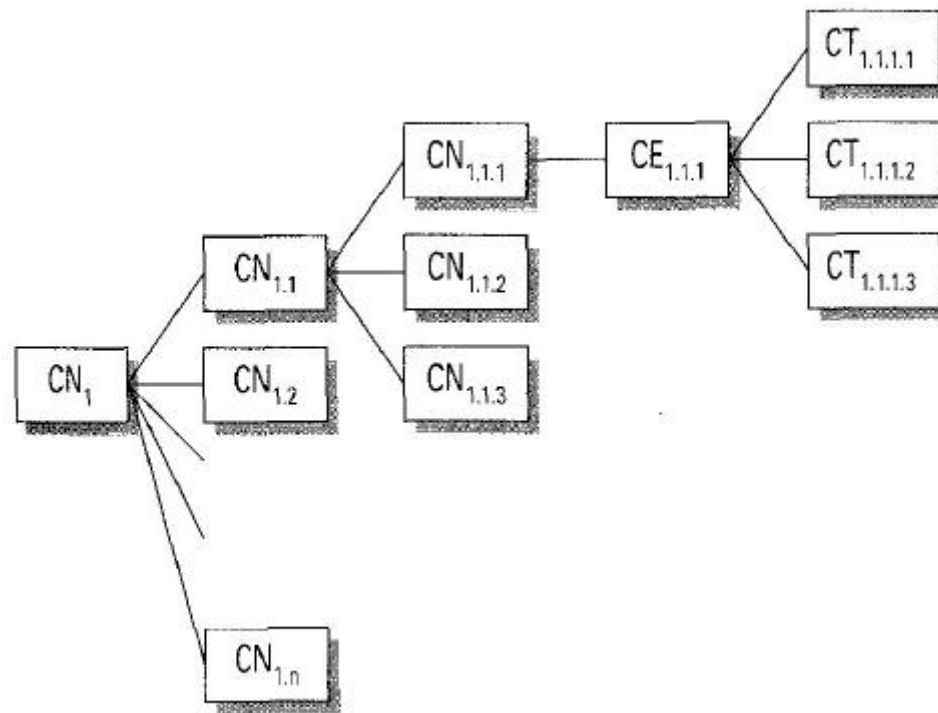
Esto es, «el contenido de información de cada especificación de caja basta para definir su refinamiento, sin depender de la implementación de ninguna otra caja».

Esto capacita al analista para desglosar jerárquicamente el sistema, pasando de la representación esencial situada en la parte superior, hasta los detalles específicos de la implementación situados en la parte inferior.

Caja negra. Esta caja especifica el comportamiento del sistema, o de parte de un sistema. El sistema (o parte de él) responde a estímulos específicos (sucesos) mediante la aplicación de un conjunto de reglas de transición que hacen corresponder el estímulo con la respuesta.

Caja de estado. Esta caja encapsula los datos de estados y de servicios (operaciones) de forma análoga a los objetos. En esta vista de especificación, se representan las entradas de la caja de estados (los estímulos) y sus salidas (las respuestas). La caja de estados también representa la «historia de estímulos» de la caja negra, es decir, los datos encapsulados en la caja de estado que deben ser mantenidos entre las transiciones implicadas

Caja limpia. Las funciones de transición que están implicadas en la caja de estado se definen en la caja limpia. Dicho literalmente, la caja limpia contiene el diseño procedimental correspondiente a la caja de estados.



A medida que se va realizando cada uno de estos pasos de refinamiento, se produce también una verificación de la corrección. Se verifican las especificaciones de las cajas de estado para asegurar que todas y cada una de ellas se ajustan al comportamiento definido por la especificación de la caja negra predecesora.

De manera similar, se verifican las especificaciones de las cajas transparentes con respecto a la caja de estados predecesora.

Es preciso tener en cuenta que los métodos de especificación basados en métodos formales se pueden utilizar en lugar del enfoque de especificación de estructura de cajas. El único requisito es que se puede verificar formalmente cada uno de los niveles de especificación.

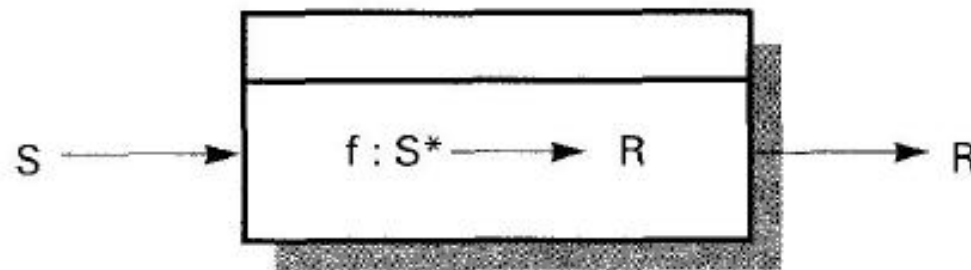
Especificación de caja negra

Una especificación de caja negra describe una abstracción, estímulos y respuestas.

La función f se aplica a una secuencia, S^* , de entradas (estímulos) y esta función los transforma en una salida (respuesta), R . **Para componentes sencillos de software, f ; puede ser** una función matemática, pero en general, f se describe empleando el lenguaje natural (o lenguaje formal).

Muchos de los conceptos introducidos para sistemas orientados a objetos son aplicables también para la caja negra. Las abstracciones de datos y las operaciones que manipulan estas abstracciones, se ven encapsuladas por la caja negra.

Al igual que una jerarquía de clases, la especificación de caja negra puede mostrar las jerarquías de utilización en que las cajas de nivel inferior heredan las propiedades de las cajas de nivel superior dentro de la estructura de árbol.



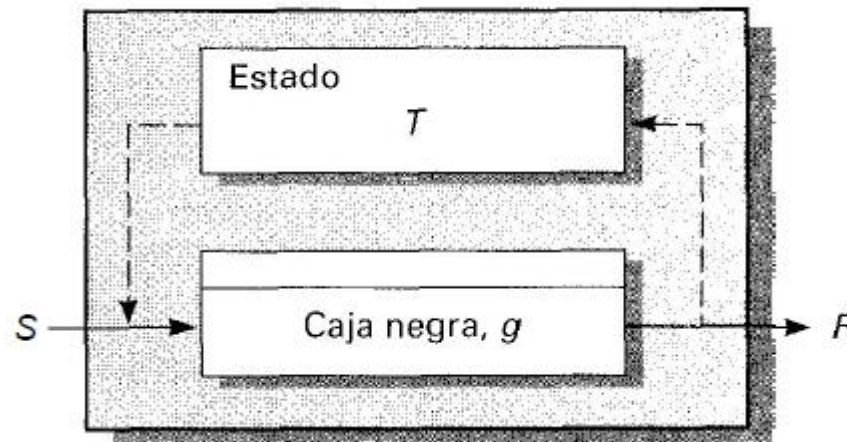
Especificación de caja de estado

La caja de estado es «una generalización sencilla de una máquina de estado». Recordando la descripción del modelado de comportamiento y de los diagramas de transición de estados, un estado es algún modo observable de comportamiento del sistema.

A medida que se produce el procesamiento, el sistema va respondiendo a sucesos (estímulos) efectuando a una transición que parte del estado y llega a algún nuevo estado.

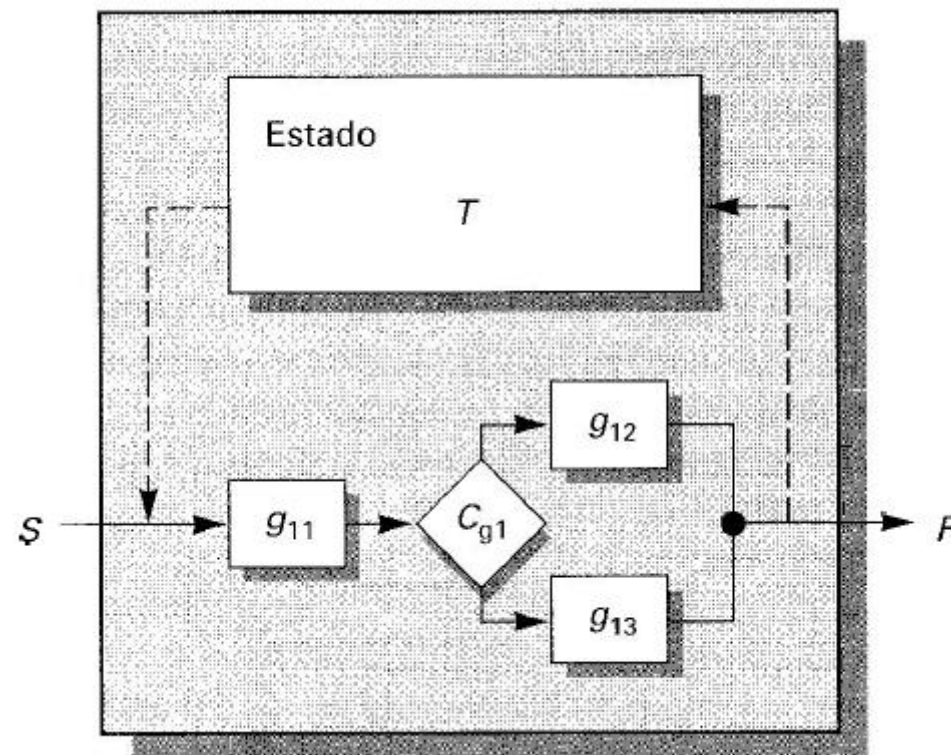
A medida que se efectúa la transición, puede producirse una acción.

La caja de estado utiliza una abstracción de datos para determinar la transición al estado siguiente (respuesta) que se producirá como consecuencia de la transición.



Especificación de caja limpia

La especificación de caja limpia está íntimamente relacionada con el diseño de procedimientos y con la programación estructurada. En esencia, la sub función g , que se encuentra dentro de la caja de estado, se ve sustituida por las estructuras de programación estructurada que implementa g .



REFINAMIENTO Y VERIFICACIÓN DEL DISEÑO

El enfoque de diseño que se utiliza en la ingeniería del software de sala limpia hace mucho uso de la filosofía de la programación estructurada.

Las funciones básicas de procesamiento se refinan ahora utilizando una «expansión progresiva de funciones matemáticas en estructuras de conectividad lógica [por ejemplo, *si-entonces-sino*] y sub funciones, donde la expansión [se] efectúa hasta que todas las funciones identificadas pudieran ser enunciadas directamente en el lenguaje de programación utilizado para la implementación.

El enfoque de la programación estructurada se puede utilizar de forma eficiente para refinar la función, pero ¿qué pasa con el diseño de datos?

En este aspecto, entra en juego un cierto número de conceptos fundamentales de diseño. Los datos del programa se encapsulan como un conjunto de abstracciones a las cuales prestan servicio las sub funciones.

Los conceptos de encapsulamiento de datos, ocultamiento de información y los tipos de datos se utilizan entonces para crear el diseño de datos.

Refinamiento y verificación del diseño

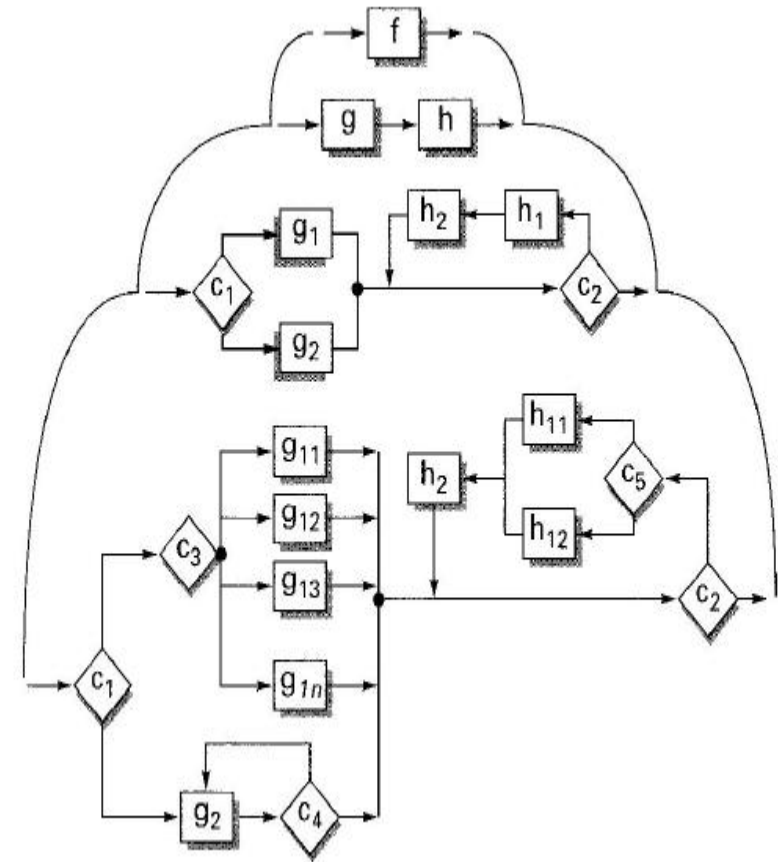
Cada especificación de caja limpia representa el diseño de un procedimiento (sub función) necesario para efectuar una transición de caja de estado.

Mediante la caja limpia, se utilizan las estructuras de programación estructurada y de refinamiento progresivo.

Una función de programa, f ; se refina para dar lugar a una sucesión de subfunciones g y h .

Estas a su vez se refinan para formar estructuras condicionales (si-entonces-sino y hacer-mientras).

Un refinamiento posterior ilustra la elaboración lógica continuada.



Ventajas de la verificación del diseño

La verificación de corrección rigurosa de cada uno de los refinamientos del diseño de caja limpia posee un cierto número de ventajas evidentes.

Se reduce la verificación a un proceso finito. La forma anidada y secuencial, en que se organizan las estructuras de control en una caja limpia, define de manera natural una jerarquía que revela las condiciones de corrección que es preciso verificar. Un axioma de sustitución nos permite reemplazar las funciones objetivo por sus refinamientos de estructura de control dentro de la jerarquía de sub demostraciones.

Es imposible asociar una importancia excesiva al efecto positivo que posee sobre la calidad la reducción de la verificación a un proceso finito.

Aun cuando todos, salvo los programas más triviales, muestran un conjunto, que en esencia es infinito, de rutas de ejecución, se pueden verificar en un número finito de pasos.

Ventajas de la verificación del diseño

Permite que los equipos de sala limpia verifiquen todas las líneas de diseño y de código. Los diseñadores pueden efectuar la verificación mediante un análisis y debate en grupo de las bases del teorema de corrección y pueden producir pruebas escritas cuando se necesite una confianza adicional en algún sistema crítico para vidas o misiones.

Da lugar a un nivel de defectos próximo a cero. Durante una revisión en equipo, se verifica por turnos la corrección de todas y cada una de las estructuras de control. Cada miembro del equipo debe estar de acuerdo en que cada condición es correcta, por tanto, un error solamente es posible si todos y cada uno de los miembros del equipo verifican incorrectamente una condición. El requisito de acuerdo unánime basada en las verificaciones individuales de resultados da lugar a un software que posee pocos o ningún defecto antes de su primera ejecución.

Ventajas de la verificación del diseño

Es escalable. Todo sistema de software, independientemente de su tamaño, posee unos procedimientos de caja transparente del más alto nivel formados por estructuras de secuencia, alternancias e iteraciones.

Cada uno de estos invoca típicamente a un gran subsistema que posee miles de líneas de código -cada uno de estos subsistemas posee su propio nivel superior de funciones y procedimientos finales-.

Por tanto, las condiciones de corrección para estas estructuras de control de alto nivel se verifican de la misma forma en que se procede con las estructuras de bajo nivel.

Las verificaciones de alto nivel pueden requerir, y merecerá la pena, una mayor cantidad de tiempo, pero no se necesita más teoría.

Ventajas de la verificación del diseño

Produce un código mejor que la comprobación unitaria. La comprobación unitaria solamente comprueba los efectos de ejecutar vías de pruebas seleccionadas entre muchas vías posibles.

Al basar la verificación en la teoría de funciones, el enfoque de sala limpia puede verificar todos y cada uno de los posibles efectos sobre los datos, porque aun cuando un programa pueda tener múltiples vías de ejecución, solamente posee una función.

La verificación es, además, más eficiente que la comprobación unitaria.

La mayor de las condiciones de verificación se pueden verificar en unos pocos minutos, pero las comprobaciones unitarias requieren una cantidad notable de tiempo para prepararlas, ejecutarlas y comprobarlas.

PRUEBA DE SALA LIMPIA

La táctica y estrategia de la prueba de sala limpia es algo fundamentalmente distinto de los enfoques convencionales de comprobación. Los métodos convencionales derivan de casos de prueba para descubrir errores de diseño y de codificación. El objetivo de los casos de prueba de sala limpia es validar los requisitos del software mediante la demostración de que una muestra estadística de casos prácticos se han ejecutado con éxito.

Prueba estadística de casos prácticos

El usuario de un programa de computadora no suele necesitar comprender los detalles técnicos del diseño.

El comportamiento visible para el usuario de ese programa está controlado por las entradas y sucesos que suelen ser producidos por el usuario. Pero en casos complejos, el espectro posible de entradas y sucesos (esto es, los casos prácticos) pueden ser extremadamente variables. ¿Cuál es el subconjunto de casos prácticos que verifica adecuadamente el comportamiento del programa?

Ésta es la primera cuestión que aborda la prueba estadística de casos prácticos.

La prueba estadística de casos «equivale a probar el software en la forma en que los usuarios tienen intención de utilizarlo».

Para lograr esto, los *equipos de prueba de sala limpia* (también llamados *equipos de certificación*) deben determinar la distribución de probabilidad de utilización correspondiente al software. La especificación (caja negra) de cada incremento del software se analiza para definir un conjunto de estímulos (entradas o sucesos) que pueden dar lugar a que el software modifique su comportamiento.

Basándose en entrevistas con posibles usuarios, en la creación de escenarios de utilización y en una comprensión general del dominio de la aplicación, se asigna una probabilidad de utilización a cada uno de los estímulos.

Certificación

En el contexto del enfoque de la ingeniería del software de sala limpia, la *certificación* implica que la fiabilidad (medida por el tiempo mínimo de fallo, TMDF) podrá ser especificada para cada componente.

El posible impacto de los componentes de software certificables va más allá de un sencillo proyecto de sala limpia. Los componentes de software reutilizables se pueden almacenar junto con sus escenarios de utilización, con los estímulos del programa y con las correspondientes distribuciones de probabilidad.

Cada uno de los componentes dispondrá de una fiabilidad certificada dentro del escenario de utilización y dentro del régimen de comprobación descrito. Esta información es sumamente valiosa para otras personas que tengan intención de utilizar estos componentes.



El enfoque de la certificación implica cinco pasos:

1. Es preciso crear escenarios de utilización.
2. Se especifica un perfil de utilización.
3. Se generan casos de prueba a partir del perfil.
4. Se ejecutan pruebas y los datos de los fallos se *registran y analizan*.
5. *Se calcula y se certifica la fiabilidad.*

La certificación para la ingeniería del software de sala limpia requiere la creación de tres modelos:

Modelo de muestreo. La comprobación de software ejecuta *m casos de prueba aleatorios*, y *queda certificada* si no se produce ningún fallo o si se produce un número de fallos inferior al especificado. El valor de *m* se deriva matemáticamente para asegurar que se alcance la fiabilidad necesaria.

Modelo de componentes. Es preciso certificar un sistema compuesto por *n componentes*. El modelo de componentes capacita al analista para determinar la probabilidad de que falle el componente *i antes de finalizar el programa*.

Modelo de Certificación. Se estima y certifica la fiabilidad global del sistema. Al final de la prueba estadística de utilización, el equipo de certificación posee la información necesaria para proporcionar un software que tenga un TMEF certificado que se habrá calculado empleando todos estos modelos. Una descripción detallada del cálculo de los modelos de muestreo, de componentes y de certificación va más allá del alcance de este libro.